Project 1: RAG and Context Engineering

Team ID: 51

Albert Chen
University of California, San Diego

Ajay Partha
University of California, San Diego

1 Data Creation Methodology

To design our golden set of Q&A pairs, we first batched the 31 source doc files into 10 batches. When creating these batches, the start of a new file was denoted by the source file name. Additionally, each line was prepended with its respective source file line number. Each of these batches was passed into ChatGPT, where it was prompted to create two Q&A pairs based on the source docs information in that batch. The initial prompt was not specific in what question types we desired, we gave the LLM flexibility in creating the examples. After all 20 Q&A pairs were created, we manually verified the quality of the pairs and gauged the noticed that comparative Q&A examples were lacking, so we crafted such examples and replaced some of the LLM-generated examples.

2 Final Pipeline Architecture

After rounds of experimentation, we were able to settle on a final pipeline design. The pipeline replaces a generic text splitter with a custom line-aware token chunker. The `LineAwareTokenRSTLoader` reads the files line by line, then creates token-budgeted chunks, and finally stores the filename, start line, and end line in each chunk's metadata. This was a critical realization because the assignment needs answers to include exact source spans, and the basic text splitter made it hard to recover the real line numbers.

The final pipeline uses 512-token chunks with 96-token overlap along with an embedding model, BAAI/bge-large-en-v1.5, which is a Sentence-Transformers model on HuggingFace. We normalize the embeddings, set the batch size to 32, and use a FAISS vector store for dense retrieval. For our retrieval method we use top-k similarity with a k-value of 25. We choose to perform reranking using BAAI/bge-reranker-large, selecting the top chunk (top_n=1) after reranking. (We would like to note that when using these embedding and reranker models, running the pipeline end to end had a runtime of roughly 20 minutes. If this runtime exceeds what is necessary, we can fallback to using the base size models. When using the base size, the runtime was roughly 8 minutes) The generator model was api-gpt-oss-120b and was given a final prompt instructing it to answer strictly from the provided documents, return "Information not found" when the answer was unsupported, and output sources in the correct format.

The context was assembled using the resources stated above. The process started by retrieving a larger candidate set from FAISS, reranking it with the cross-encoder model, and then passing the top chunks into the generator. This was able to balance the context budget we were given with recall. The larger k value of 25 allowed the retriever multiple chances to find the right documentation, whereas the smaller top_n value of 1 stopped the generator prompt from containing too much noise. Because the serialized context chunks had line spans, this allowed the model to cite the correct sources directly. The final prompt remained compact because it chose only 1 chunk of at most 512 tokens, ensuring our context stayed under our 2,000 token budget.

38th Conference on Neural Information Processing Systems (NeurIPS 2024) Creative AI Track.

3 Experimentation Methodology

We used RapidFire AI to compare different RAG pipeline configurations instead of changing one setting at a time. The thought process behind the experiments was to see which configuration knobs had the greatest impact on retrieval quality and final answer generation. The task at hand required accurate answers and correct sources, so we evaluated configurations using retrieval metrics including F1, precision, recall, and overall retrieval score.

The main configuration knobs that were tuned were chunk size, overlap, retrieval depth `k`, search type, embedding model, reranker model, and reranker cutoff `top_n`. These knobs control the main tradeoffs in a RAG system. Size of chunks along with overlap determine how the documentation is going to be split before embedding. Larger chunks provide better context but introduce more noise, while smaller chunks improve retrieval precision but lack context. Retrieval depth `k` decides how many candidate chunks are gotten from FAISS before the reranking process. Again with this, there is a tradeoff with a larger k bringing more context but also with more noise and token cost. Then there is search type which tests if standard similarity search or MMR is better for the documentation task. The embedding model controls how accurately questions and documentation chunks are represented in the vector space. The reranker model regulates how well the retrieved candidates are reordered. Lastly, the reranker cutoff `top_n` directs how many reranked chunks are passed nto the generator model which affects the context budget and noise in the final prompt.

Our process involved a grid search-style experimentation with RapidFire's multi-config API. The evaluations were run across 4 shards of data. For each group of tests, we set values for the knobs and let RapidFire evaluate the configurations on the golden Q&A set. Each configuration combined different knob setups to see which knobs affected our scores the most. After running the evaluation pipeline, RapidFire AI was able to provide us comparable metrics across runs. This allowed us to test different RAG design choices while keeping certain features fixed.

In Table 1 we report the 12 most important runs. We focus on the runs that involved the most important changes. These configurations show the progression of our RAG pipeline and the important decisions that improved our score.

Table 1: RapidFire AI configuration knobs.

ID	Chunk Size	Overlap	k	Search Type	Embedding Model	Reranker Model	top_n
1	384	64	10	similarity	`all-MiniLM-L6-v2`	`ms-marco-MiniLM-L6-v2`	3
2	384	64	10	similarity	`all-MiniLM-L6-v2`	`ms-marco-MiniLM-L6-v2`	5
3	384	64	20	similarity	`all-MiniLM-L6-v2`	`ms-marco-MiniLM-L6-v2`	3
4	512	96	20	similarity	`all-MiniLM-L6-v2`	`ms-marco-MiniLM-L6-v2`	2
5	512	96	25	similarity	`all-MiniLM-L6-v2`	`ms-marco-MiniLM-L6-v2`	2
6	512	96	20	MMR	`all-MiniLM-L6-v2`	`ms-marco-MiniLM-L6-v2`	2
7	896	128	25	similarity	`all-MiniLM-L6-v2`	`ms-marco-MiniLM-L6-v2`	2
8	896	128	25	similarity	`all-MiniLM-L6-v2`	`BAAI/bge-reranker-base`	2
9	896	128	25	similarity	`BAAI/bge-base-en-v1.5`	`BAAI/bge-reranker-base`	2
10	512	64	25	similarity	`BAAI/bge-large-en-v1.5`	`BAAI/bge-reranker-base`	2
11	512	128	25	similarity	`BAAI/bge-large-en-v1.5`	`BAAI/bge-reranker-large`	2
12	512	96	25	similarity	`BAAI/bge-large-en-v1.5`	`BAAI/bge-reranker-large`	2
13	512	96	25	similarity	`BAAI/bge-large-en-v1.5`	`BAAI/bge-reranker-large`	1

These 13 configurations are meaningful because they each test a different hypothesis about the RAG pipeline. Run IDs 1-3 test the effects of retrieval depth and context size in the initial setup. Run IDs 4-6 test if 512 token chunk size is better than the initial 384, along with how does MMR compare with similarity. Run IDs 7-9 test the effect of different embedding and reranking models. Finally run IDs 10-13 involve tuning these knobs to find the best pipeline with these tradeoffs in mind. The last run also involves top_n = 1 which we found to be very useful in our precision.

4 Results and Analysis

Table 2: Results for representative RapidFire AI configurations on 4 shards of data.

ID	F1	Precision	Recall	Retrieval Score	Gen Score
1	0.335	0.233	0.625	0.398	0.903
2	0.226	0.140	0.625	0.330	0.860
3	0.385	0.267	0.725	0.459	0.917
4	0.558	0.450	0.775	0.594	0.920
5	0.558	0.450	0.775	0.594	0.960
6	0.558	0.450	0.775	0.594	0.957
7	0.500	0.375	0.750	0.541	0.883
8	0.566	0.425	0.850	0.613	0.906
9	0.600	0.450	0.900	0.650	0.973
10	0.625	0.500	0.875	0.667	0.940
11	0.683	0.550	0.950	0.727	0.986
12	0.683	0.550	0.950	0.727	0.983
13	0.783	0.800	0.775	0.786	0.876

The results show an obvious progression from our initial MiniLM-based retrieval pipeline to the advanced BGE-based configurations. The earlier runs performed the worst due to the usage of lower retrieval depth, and weaker embedding/reranking models. The first run used 384-token chunks, 64-token overlap, k=10, and top_n=3 which is important to note as experiments were really built off of this. The run achieved an F1 score of 3.35 and a retrieval score of 0.398. Run 2 kept all knobs the same but increased top_n from 3 to 5. This made performance significantly worse, dropping both F1 and precision to 0.226 and 0.140 respectively. This showed that adding more chunks into the generator didn't equal improved performance, it actually just introduced noisy context.

Run 2 to run 3 shows how increasing retrieval depth improved efficiency. Run 3 increased k from 10 to 20 and saw recall jump from 0.625 to 0.725 and F1 increase from 0.335 to 0.385. This was a significant realization but clearly there were still some knobs such as chunking and certain models that were stunting performance.

Runs 4 and 5 demonstrated that a slightly larger chunk size, 512 with 96 token overlap, provided a much better baseline. This change in chunking and another decrease in top_n to 2 took retrieval score to the 0.5's. However, as seen in Run 7, we found that further increasing the chunk size and overlap to 896 and 128 respectively did not continue to improve results, as it led to a worse retrieval and generation score.

Run 6 tested a new search type, MMR, against similarity search which is what we had been using. The result showed us that MMR achieved the same F1 score and retrieval score with a very similar generation score. These results were corroborated with additional search ablations not included in the table. With MMR search not providing an improvement over similarity search, we decided to remain with using similarity search.

Runs 8 and 9 tested the difference of stronger embedding and reranking models. Run 8 had the same knobs as Run 7 but replaced the reranker with BAAI/bge-reranker-base. This replacement improved F1 from 0.500 to 0.566 and recall from 0.750 to 0.850, showing how critical this was. Run 9 then changed the embedding model to BAAI/bge-base-en-v1.5 which improved F1 to 0.600, recall to 0.900, and retrieval score to 0.650. Previously we were getting stuck running different combinations of the other knobs and not seeing any noticeable difference, this is when our pipeline had one of the biggest jumps in performance.

Runs 10 through 13 were our strongest configurations, making use of BGE embeddings and reranking. Run 10 introduced a larger BAAI/bge-large-en-v1.5 embedding model which improved the precision of retrieval. Run 11 introduced a larger BAAI/bge-reranker-large reranker model which improved the precision and recall score. Run 12 tried a smaller overlap size of 96, which kept the retrieval metric scores the same and caused a slightly lower generation score. Noticing that our configuration setup was leading to high recall but low precision scores, our pipeline was seemingly able to correctly retrieve the relevant context but also retrieve unnecessary context. Thus, we believed that reducing top_n again could improve the precision of our pipeline. Run 13 experimented with a top_n of

1, resulting in a significant jump in precision and a lower recall, ultimately leading to the highest retrieval score. However, this led to a lower generation score.

Table 3: Results for configurations 12 and 13 on 12 shards of data.

ID	top_n	F1	Precision	Recall	Retrieval Score	Gen Score
12	2	0.700	0.550	1.000	0.750	0.970
13	1	0.933	0.950	0.925	0.936	0.936

Following the aforementioned runs, we were especially interested in the effects of using a top_n of 2 vs 1. When running the experiments in Table 2 we used 4 shards of data, which could lead to a level of randomness in the results. In order to reduce randomness we isolated Runs 12 and 13 and reran those configurations for 12 shards. The full results of these experiments can be seen in Table 3. Ultimately, we see that the use of top_n = 1 in configuration 13 led to a noticeably higher retrieval and generation score, leading us to finalize on setting top_n = 1 for our final pipeline architecture.

If given more time, we would be interested in exploring techniques to improve the generation quality of our RAG pipeline. We found that with our final pipeline architecture, we had reached a point where the retrieval score was higher than the generation score. This implies that while our retrieval method may be retrieving the necessary context to give a quality answer, the actual answer generated by the LLM using this context was not sufficient. So, we believe that exploring context stitching techniques could lead to an improvement in generation quality. And, we are curious if additional prompt engineering could be beneficial.

5 AI Tools Statement

While working on this project, LLMs such as Gemini and ChatGPT were used to provide ideas for directions to explore regarding the pipeline knobs. In particular, AI was useful in devising the custom line-aware token chunker.